ORBIT-LLM — Findings Log

Running record of experimental results. Each section states what was measured, how, what was found, and what is still open. Raw data lives in `results/`.

Hardware: Intel Core i5-1335U (2 P-cores + 8 E-cores, 12 threads), Intel Iris Xe iGPU, 15.7 GB RAM, Windows 11. **Software:** OpenVINO 2026.3, OpenVINO GenAI 2026.3 (built from source at `bd8d6542`). **Model:** TinyLlama-1.1B-Chat-v1.0, exported with `optimum-cli` to FP16, INT8 and INT4 IR.

Precision	Size on disk	Folder
FP16	2098.2 MB	`tinyllama-ov`
INT8	1050.7 MB	`tinyllama-int8`
INT4	623.6 MB	`tinyllama-int4` (≈12% of layers kept at INT8)

Phase 2 — Benchmarking harness

Method

Every configuration runs in a **fresh process** (`run_matrix.py` → `benchmark_matrix_worker.py`), so one model's memory and caches cannot leak into the next measurement.

Per configuration: load the pipeline, **1 unmeasured warm-up**, then **3 measured runs**. Generation is made deterministic and length-exact:

- `do_sample = False` (greedy), `ignore_eos = True`, `max_new_tokens = N` → exactly N output tokens.
- Prompts are built by tokenizing repeated text with `max_length` + `truncation`, so the model sees exactly the requested number of input tokens. Both counts are verified against OpenVINO's own `perf_metrics` and the run is marked failed if they differ.
- `apply_chat_template = False`, so the template cannot add hidden tokens.

Metrics come from OpenVINO `perf_metrics` (TTFT, TPOT, throughput, generate duration). Peak memory is the process working set, sampled every 10 ms by a background thread during generation. Note that OpenVINO's reported throughput is decode throughput (≈ 1000 / TPOT); it excludes TTFT.

Baseline (27 input tokens → 50 output tokens, `benchmark_one.py`)

Mean of 3 runs	FP16	INT8	INT4
TTFT	527 ms	469 ms	364 ms
TPOT	115.4 ms	80.5 ms	46.3 ms
Throughput	8.83 tok/s	12.53 tok/s	21.66 tok/s
Peak RSS	3972 MiB	2126 MiB	1338 MiB
vs FP16	1.00×	1.42× throughput, −46.5% RSS	2.45× throughput, −66.3% RSS

FP16 throughput varied 7.5–10.4 tok/s across just 3 runs, which was the first sign of the noise problem described in Finding 4.

Phase 3 — Quantization workload matrix

3 precisions × input {32, 128, 512} × output {32, 128} = 18 configurations × 3 runs.

Finding 1 — The first matrix measured the prefix cache, not prefill

In the first matrix (`results/matrix_summary.csv`), the same prompt was used for warm-up and every measured run. TTFT came out **flat at ~45–105 ms regardless of prompt length**, for example FP16 at 512 input tokens: 106 ms. A 512-token prefill cannot be as fast as a 32-token one, so the prompt was not being processed again.

`test_prefix_cache.py` confirmed it: a repeated 512-token prompt was fast, and a new prompt of the same length was slow.

Mechanism (confirmed in the GenAI source). On CPU, `LLMPipeline` defaults to the paged-attention / continuous-batching backend and builds it with `get_latency_oriented_scheduler_config()` (`src/cpp/src/utils.cpp`). That configuration sets `enable_prefix_caching = true` and an unlimited `max_num_batched_tokens`. KV-cache blocks from earlier requests are reused when a new prompt shares the same prefix, so repeated prompts skip almost all of prefill.

Consequence. That matrix is a valid measurement of *cache-hit* TTFT and decode speed, but not of prefill cost. It is kept as `matrix_*` (cached) data.

Pitfall for later phases. Passing a plain `SchedulerConfig{}` to the pipeline does *not* keep these defaults. It sets `enable_prefix_caching = false` and `max_num_batched_tokens = 256`, which splits a long prefill into 256-token chunks. Any KV-cache experiment must start from the latency-oriented values. `orbit_run` does this.

Finding 2 — Cold-prompt matrix (`results/matrix_cold_summary.csv`)

`benchmark_cold_worker.py` gives warm-up and each measured run a **different** prompt (different topic, same exact length), so every measured run pays full prefill.

Median results:

Model	In	Out	TTFT ms	TPOT ms	Tok/s	Peak MiB	Speedup vs FP16
FP16	32	32	801.6	118.1	8.47	4147	1.00×
FP16	128	32	2708.6	124.9	8.01	4112	1.00×
FP16	512	32	10506.1	115.6	8.65	4165	1.00×
INT8	32	32	404.5	63.5	15.75	2124	1.86×
INT8	128	32	1110.6	61.2	16.35	2138	2.04×
INT8	512	32	3719.7	57.9	17.27	2191	2.00×
INT4	32	32	316.6	45.9	21.81	1337	2.58×
INT4	128	32	1185.0	45.1	22.17	1347	2.77×
INT4	512	32	5689.1	52.9	18.89	1401	2.18×

The full 18-row table, including output length 128, is in `matrix_cold_summary.csv`.

- **TTFT now scales with prompt length**, roughly linearly. For FP16, 16× more input tokens (32 → 512) gives 13× higher TTFT.
- **Memory savings are stable:** INT8 saves 47–49% and INT4 saves 66–68%, at every workload size.
- **Decode speedup:** INT8 is 1.8–2.0× faster than FP16, INT4 2.2–3.1× faster.

Finding 3 — Prefill and decode prefer different precisions

Prefill rate at 512 input tokens is input tokens / TTFT. Decode rate is 1000 / TPOT.

	FP16	INT8	INT4
Prefill (512 tokens)	49 tok/s	**138 tok/s**	90 tok/s
Decode	8.6 tok/s	17.3 tok/s	**18.9 tok/s**

INT4 decodes fastest but INT8 prefills fastest. At 512 input tokens, INT4's TTFT is 53% higher than INT8's (5.7 s vs 3.7 s).

Likely explanation (hypothesis, not yet verified): - Decode is **memory-bandwidth-bound**. Each token streams all weights once, so smaller weights win, and INT4 is fastest. - Prefill processes many tokens per weight load, so it is **compute-bound**. INT4 has to unpack and dequantize 4-bit groups, which adds compute exactly where compute is the bottleneck.

If this holds, it is direct evidence for the plan's phase-aware research question: the best configuration would depend on whether a request is prefill-heavy (long prompt, short answer) or decode-heavy (short prompt, long answer).

> **Status: not reproduced.** In the C++ re-run (below), INT4's TTFT at 512 input tokens was 3.9 s / 3.1 s (32 / 128 output tokens) against INT8's 3.2 s / 3.6 s. That is no consistent penalty. The original 5.7 s was most likely inflated by the same noise as Finding 4. Treat this as an open hypothesis. Test it with longer prompts (1k–2k tokens), more runs and interleaved ordering before claiming it.

Finding 4 — Run-to-run and day-to-day noise is large

- The original INT4 512→128 runs (`matrix_cold_original.csv`) reported TPOT of 116–179 ms/token, against ~49 ms for every other INT4 configuration. That configuration was rerun at 13:11 (`int4_512_128_retest.csv`), and the rerun rows replace the originals in `matrix_cold_clean.csv`. **This is the only difference between the original and clean files.**
- Decode speed for identical work differed between days. FP16 TPOT was 87–94 ms in the cached matrix (Oct 4) and 109–125 ms in the cold matrix (Oct 5). Decode work is the same in both, so the two matrices can be compared on **TTFT only**, not on decode speed.
- Likely causes: background load, OneDrive sync (the project folder is on OneDrive) and thermal state of a 15 W laptop CPU.

Action taken. `orbit_run` records available RAM and system CPU-busy % immediately before every measured run, so noisy runs can be identified instead of guessed at.

Open questions

1. **RSS is about 2× the on-disk size for every precision:** FP16 4.15 GB vs 2.10 GB, INT8 2.13 GB vs 1.05 GB, INT4 1.34 GB vs 0.62 GB. - **FP16 — explained.** The CPU plugin on this machine reports `INFERENCE_PRECISION_HINT = f32` and capabilities `FP32, INT8`: there is no native FP16 or BF16 compute on the i5-1335U. FP16 weights are therefore converted to f32 at compile time. 1.1B parameters × 4 bytes ≈ 4.1 GiB, which matches the measured RSS. On this CPU, the "FP16" baseline is really *f32 compute*. - **INT8 and INT4 — explained in Phase 4 (Finding 9).** Committed private memory after warm-up is only 850 MiB for INT4, while the working set is 1350 MiB. The ~500 MiB difference is the memory-mapped IR file, which the working set counts and the commit charge does not. The weights are resident twice: once as the mapped file, once as the compiled model's private copy.
2. **Prefill is slow even for INT8** (138 tok/s). Does it improve with P-cores only, or a different thread count? Early `orbit_run` checks suggest P-core-only may beat the 12-thread default on this hybrid CPU.
3. **INT4's prefill penalty:** does it grow with prompt length (1k, 2k tokens)? That decides where the INT8/INT4 crossover is.

4. **Lazy weight loading:** RSS stays at ~125–275 MB after pipeline construction and jumps only on the first generate call (see `../LOG.md`). Weights appear to be paged in on first use, which is why warm-up matters.

C++ executor parity (`orbit_run`)

From here on, experiments run through `cpp/orbit_run.exe`, which uses the OpenVINO GenAI **C++ API**. The Python layer only orchestrates and analyses. Before switching, the C++ executor was checked against the Python harness.

Full matrix re-run (`results/cpp_matrix_cold_*`)

The same 18 cold-prompt configurations were run through `run_cpp_matrix.py`: 54/54 runs succeeded in 10.4 minutes.

- **Functionally identical:** the same exact input and output token counts on every run.
- **Memory:** peak memory is 45–55 MiB lower in every configuration. That is the Python interpreter: RSS before loading is 54 MiB under Python and 8 MiB under C++.
- **Timing was mostly lower:** TTFT was 13–51% lower in most configurations, and FP16 TPOT 9–23% lower. Some INT4 configurations were slower, with TPOT up to 20% higher.
- **Conditions were not the same:** the Python matrix ran in the morning with applications open; the C++ matrix ran in the evening after closing them. Background CPU during the C++ run: median 28.5%, range 17–57%. Available RAM was only 1.0–1.5 GB during the FP16 configurations.

Interleaved A/B (same session, alternating executors)

To separate "C++ vs Python" from "morning vs evening", two configurations were run alternately: Python, C++, Python, C++, … for 3 rounds × 3 measured runs each (n = 9 per executor). Raw data is in `results/ab_python_vs_cpp_*`. Medians:

Configuration	Executor	TTFT	TPOT	Peak RSS
FP16, 128 in → 32 out	Python	1612 ms	126.4 ms	4113 MiB
	C++	1496 ms	133.7 ms	4064 MiB
INT8, 128 in → 128 out	Python	955 ms	62.5 ms	2138 MiB
	C++	761 ms	54.2 ms	2090 MiB

- **FP16:** no meaningful difference. TTFT is 7% lower and TPOT 6% higher, and the run ranges overlap heavily.

- **INT8:** C++ is 20% lower on TTFT and 13% lower on TPOT, and much less variable. C++ TTFT range was 707–860 ms; Python's was 591–1144 ms. A possible cause is the Python memory-sampling thread competing for the GIL and a core. Not verified.
- **Conclusion:** the C++ executor is a faithful replacement. Its numbers are equal to or slightly better than Python's. The large gaps in the full-matrix comparison came mostly from different machine conditions, not from the executor.

Finding 5 — Absolute numbers drift by up to ~40% between sessions

FP16 TPOT for comparable work was 92 ms in the C++ matrix and 126–134 ms in the A/B test an hour later, on the same evening, with the same binary. FP16 is the configuration closest to this machine's free-RAM limit, so it is the most sensitive.

Rules adopted for every later experiment:

1. Compare configurations only **within the same session**. Interleave or randomise their order, rather than running all of A and then all of B.
2. Use **5 measured runs** per configuration instead of 3, and report the median with its range.
3. Keep the per-run machine state (`sys_available_ram_mib`, `sys_cpu_busy_percent`) in the dataset. For the Performance Twin (Layer 1) these are **input features**, not noise to discard. Predicting performance under a given machine state is exactly the twin's job.
4. Pause OneDrive sync before benchmarking. The project folder is synced, and the sync client used ~16% CPU while results were being written.

Phase 4 — KV cache and memory (`plans/kv_precision.json`, `prefix_cache.json`, `cache_size.json`)

Method: `run_experiment.py`, all through `orbit_run`. Each plan runs 2 rounds in shuffled order, with 3 measured runs per round, so n = 6 per configuration. 252 measured runs, 0 failures, 69 minutes in total. Raw rows are in `results/experiments/*.jsonl`; summaries are in the matching `*_summary.csv`. Background CPU before runs: median 25%, range 9–76%.

Defaults verified on this machine (`Core.get_property("CPU", …)`): `KV_CACHE_PRECISION = u8`, `INFERENCE_PRECISION_HINT = f32`, `INFERENCE_NUM_THREADS = 0` (auto), `SCHEDULING_CORE_TYPE = ANY_CORE`. OpenVINO already quantizes the KV cache to 8 bits by default on this CPU.

Expected KV size for TinyLlama (22 layers, 4 KV heads, head dim 64): 2 × 22 × 4 × 64 = 11,264 values per token. That is 11.0 KiB at u8, 22.0 KiB at f16 and 44.0 KiB at f32. At 1920 tokens: 21 / 41 / 83 MiB.

Finding 6 — KV precision behaves exactly as theory predicts, but barely matters at 1.1B

Peak RSS growth from 256 to 1920 prompt tokens (+1664 tokens), median of 6 runs:

KV type	INT4	INT8	Extra vs u8 (measured)	Extra vs u8 (theory)
u8 (default)	+222.6 MiB	+223.3 MiB	—	—
f16	+237.9 MiB	+236.5 MiB	+14 MiB	+17.9 MiB
f32	+272.8 MiB	+273.0 MiB	+50 MiB	+53.6 MiB

- The *differences* between KV types match the theoretical KV sizes within a few MiB, for both weight precisions.
- **Speed:** TTFT shows no consistent effect. At 1920 tokens, INT4 TPOT was 56.5 ms with u8, 50.0 ms with f16 and 46.5 ms with f32. That suggests dequantizing the KV cache costs some decode time at long context, but the ranges overlap and INT8 shows no such trend. Weak signal.
- **Conclusion for a 1.1B model:** at its full 2k context, the KV cache is 21–83 MiB, against ~1.3–2.1 GB of weights. KV precision is not a meaningful lever here. It becomes one for larger models and longer contexts. For example, a 3B model with 8 KV heads × head dim 128 needs ≈ 112 KiB per token at f16, so 8k tokens is about 0.9 GiB. **Phase 5 should add a 3B model.**

Finding 7 — Memory growth with prompt length is ~12× larger than the KV cache

Total growth is **0.134 MiB per prompt token**, while the u8 KV cache accounts for only 0.011 MiB per token.

Hypothesis: the paged-attention path computes **full-vocabulary logits for every prompt token** during prefill. 32,000 vocab × 4 bytes = 0.122 MiB per token; adding the 0.011 MiB of KV gives 0.133 MiB per token, against 0.134 measured. The arithmetic fits closely, but it is not yet confirmed from the source or a profiler.

The SDPA backend grows even more: +462 MiB RSS from 256 to 1920 tokens, against +222 MiB for PA (n = 1 each). Its TTFT is also 69% higher at 1920 tokens (24.4 s vs 14.4 s). 32 heads × 1920^2 × 4 bytes = 450 MiB, which suggests SDPA materializes the full attention-score matrix while PA does not. Also a hypothesis.

> **Status: logits hypothesis refuted (Finding 16).** It predicted ~0.60 MiB/token for Qwen2.5-3B, with its 151,936-token vocabulary. The measured growth was 0.23 MiB/token. The growth tracks MLP activation size instead. See Finding 16.

Why it matters: for memory prediction, prompt length has a much larger effect than the KV cache does on this model. A memory model for the Twin should be `weights + a × prompt_tokens + kv_bytes × (prompt + output tokens)`, with `a` measured per backend.

Finding 8 — Prefix caching: ~177× faster TTFT on repeated prompts, no cost otherwise

Model	Prompt	TTFT, repeated, caching on	TTFT, repeated, caching off	TTFT, unique, on	TTFT, unique, off
INT4	512	**69 ms**	4939 ms	3407 ms	3507 ms
INT4	1920	**77 ms**	12949 ms	13750 ms	13585 ms
INT8	512	**60 ms**	2643 ms	4529 ms	3386 ms
INT8	1920	**72 ms**	12737 ms	12732 ms	18955 ms

- With caching on, TTFT on a repeated prompt is almost independent of length: 60–77 ms for both 512 and 1920 tokens. Only the newly generated tokens need computing.
- For unique prompts, on and off are within noise of each other, and peak memory is identical. Leaving prefix caching on, which is the `LLMPipeline` default, costs nothing measurable for a single user.
- Noisy cells: INT8 unique/off at 1920 tokens ranged 11.4–25.4 s, and INT8 unique/on at 512 tokens ranged 2.6–6.6 s. Both are session noise, not configuration effects.
- **For ORBIT-LLM:** "does this request share a prefix with a recent one?" is the single strongest TTFT predictor found so far. Chat follow-ups and repeated system prompts get this effect for free. The Workload Profiler should detect it.

Finding 9 — Working-set memory hides real memory commitments

`cache_size_gb` pre-allocates the KV cache. In the main experiment it changed peak **working set** by only +20–56 MiB, even at 4 GB. After adding committed private memory (`PrivateUsage`) to `orbit_run`, a follow-up check (INT4, 512 tokens, n = 1 each) showed:

`cache_size_gb`	Working set after warm-up	Committed after warm-up	Peak committed
0 (dynamic)	1350 MiB	850 MiB	851 MiB
2	1352 MiB	2892 MiB (+2042)	**4323 MiB**
4	1355 MiB	4948 MiB (+4098)	**8431 MiB**

- The pre-allocated cache *is* committed in full (+2.0 / +4.0 GiB), but its pages are not touched, so the working set never shows it.
- **Peak** committed memory reaches ≈ 2× the cache size during initialization. A 4 GB cache transiently commits about 7.6 GB extra.
- The same measurement explains the INT8/INT4 "2× disk size" question (Open question 1): working set = private memory + the memory-mapped IR file.

Consequence for the Performance Twin: OOM risk must be predicted from **committed memory against the system commit limit**, not from working set. All new `orbit_run` rows record `commit_after_load_mib`, `commit_after_warmup_mib`, `commit_after_generation_mib` and `lifetime_peak_commit_mib`. The Phase 4 experiment rows predate this field.

Finding 10 — How much of the noise does machine state explain?

Each run's deviation from its configuration's median was correlated with the machine state recorded just before it (all 252 Phase 4 runs):

Metric	Typical spread (p10–p90 of run / median)	r with background CPU %	r with free RAM
TTFT	0.78 – 1.23	0.25	−0.05
TPOT	0.89 – 1.12	0.13	0.01

Background CPU load explains only a small part of the variance. Free RAM explains none of it on these sizes, which never came near the limit. The remaining noise is probably thermal and power-limit behaviour of a 15 W CPU, plus thread placement on hybrid P/E-cores. Neither is measured yet.

Implications: - (a) Expect a TTFT prediction-error floor of roughly ±20% on this laptop. Evaluate the Twin against that floor, not against zero. - (b) Record CPU frequency or temperature, and CPU load *during* the run, before training the Twin. - (c) The P-core experiment (week 5) may reduce the variance as well as the mean.

Phase 5 — Device and CPU core placement (`plans/device_cores.json`, `gpu_fp16.json`, `load_time.json`)

Method: same as Phase 4 (2 shuffled rounds × 3 runs, n = 6 per configuration), 64 output tokens, unique prompts. `orbit_run` now also records CPU behaviour *during* each run: effective MHz, own CPU share and background CPU share. 216 measured runs plus 12 load-time invocations, 0 failures, 49 minutes.

Variants:

Variant	Settings
`cpu_auto`	OpenVINO default (all 12 threads, any core)
`cpu_pcore`	`SCHEDULING_CORE_TYPE = PCORE_ONLY`
`cpu_ecore`	`SCHEDULING_CORE_TYPE = ECORE_ONLY`

Variant	Settings
`cpu_4t`, `cpu_8t`	`INFERENCE_NUM_THREADS` = 4 or 8
`gpu`	Intel Iris Xe iGPU

Finding 11 — The iGPU wins every per-request metric, by a wide margin

Medians, n = 6:

Model	Prompt	Best CPU TTFT	GPU TTFT	Best CPU TPOT	GPU TPOT	GPU decode tok/s
INT4	128	969 ms (8t)	**315 ms**	38.2 ms (P-core)	**26.6 ms**	37.5
INT4	1024	9011 ms (E-core)	**1740 ms**	46.9 ms (P-core)	**28.2 ms**	35.5
INT8	128	1013 ms (P-core)	**291 ms**	58.6 ms (P-core)	**40.5 ms**	24.7
INT8	1024	7999 ms (8t)	**1804 ms**	52.1 ms (P-core)	**37.7 ms**	26.6
FP16	128	3420 ms (auto)	**225 ms**	123.1 ms (auto)	**48.4 ms**	20.7
FP16	1024	27285 ms (auto)	**2358 ms**	171.1 ms (auto)	**49.7 ms**	20.1

- **Prefill:** the GPU is 3–5× faster than the *best* CPU configuration for INT4/INT8, and 12× faster than the CPU for FP16 at 1024 tokens.
- **Decode:** the GPU is 1.4× faster than the *best* CPU configuration (P-cores).
- **FP16 benefits most.** The iGPU has native FP16; the CPU has to compute in f32 (Open question 1). FP16 on the GPU decodes at 20 tok/s, against 6–8 tok/s on the CPU, and uses 2.5 GB RSS instead of 4.1 GB.
- **The GPU is much more stable.** INT4 at 1024 tokens: GPU TTFT ranged 1470–1772 ms (±10%), CPU default 7171–14398 ms (2×). Moving work off the contended CPU removes most of the session noise from Findings 5 and 10.
- **Caveat on GPU memory:** process RSS is *lower* on the GPU (INT4 1080 MiB vs 1307 MiB), but iGPU buffers live in shared system memory and may not all be attributed to the process working set. GPU memory needs a separate measurement before any memory claim is made.

Finding 12 — On the hybrid CPU, the OpenVINO default is never the best configuration

Two P-cores only (`cpu_pcore`) beat the 12-thread default on decode in all four cells:

Model	Prompt	TPOT, default	TPOT, P-core only	Change
INT4	128	63.4 ms	38.2 ms	−40%
INT4	1024	60.5 ms	46.9 ms	−22%
INT8	128	69.4 ms	58.6 ms	−16%
INT8	1024	84.3 ms	52.1 ms	−38%

- `cpu_pcore` used only ~15% of total CPU time (≈ 2 of 12 logical processors) and was still the fastest decoder.
- Likely mechanism (hypothesis): decode is memory-bandwidth-bound, and each layer's work is split across threads. With E-cores in the pool, every step waits for the slowest (E-core) thread to finish its share — a straggler effect. More threads add synchronization cost without adding bandwidth.
- **Prefill is different.** More threads help this compute-bound phase: 8 threads gave the best INT8 TTFT at 1024 tokens and the best INT4 TTFT at 128 tokens. **The best CPU thread placement differs between prefill and decode.** This is concrete evidence for the plan's phase-aware research question. Within one request, OpenVINO compiles one thread configuration, so exploiting it would need two compiled models or a per-phase switch. That is a design question for the controller.

Finding 13 — The model cache helps the GPU and hurts the CPU

Pipeline construction time, INT4, 3 rounds. Round 1 of a cached variant populates the cache.

Variant	Round 1	Round 2	Round 3
CPU, no cache	2.3 s	2.0 s	2.4 s
CPU, `cache_dir`	10.8 s	13.2 s	10.5 s
GPU, no cache	7.6 s	8.0 s	9.2 s
GPU, `cache_dir`	20.8 s *(cold)*	**2.1 s**	**2.8 s**

- **GPU:** without a cache, every load recompiles kernels, which took 6–11 s in `device_cores` and 14–19 s for FP16. With a warm cache, load drops to 2–3 s, about the same as the CPU. Populating the cache costs one slow load of ~21 s.
- **CPU:** the cache makes every load ~5× *slower*, even when warm (10–13 s vs 2–2.4 s). The likely reason is that the cached blob is read in full instead of memory-mapping the IR. Not verified.
- **Rule:** enable `cache_dir` for GPU, never for CPU.

Finding 14 — What this means for ORBIT-LLM's decision problem

On this laptop, for steady-state generation, **GPU + INT4 dominates**: fastest TTFT, fastest decode, lowest variance. A controller that always picked it would be near-optimal per request. The real decisions are therefore around it:

1. **Cold start vs request size.** Example: one request with a 128-token prompt and 64 output tokens. Single-run numbers, so approximate.

Path	Load	+ TTFT	+ decode	≈ Total
CPU (P-cores)	~1.4–3.7 s	1.0 s	64 × 38 ms = 2.4 s	5–7 s
GPU, no cache	6–11 s	0.3 s	1.7 s	8–13 s
GPU, warm cache	2–3 s	0.3 s	1.7 s	4–5 s

Whether the GPU is worth it depends on cache state and how much work follows. That is exactly a predict-then-choose problem. 2. **Phase-aware CPU threads** (Finding 12), when the GPU is unavailable or busy. 3. **Prefix reuse** (Finding 8), the largest single TTFT effect: ~177×. 4. **Memory and OOM limits** (Finding 9), which will start to bind with a 3B model.

Next: a 3B model is now the most important addition. It makes KV-cache and memory decisions real, and tests whether "GPU always wins" survives a model 3× larger in a GPU that shares system memory.

Layer 1 — Performance Twin v0 (`build_dataset.py`, `twin_v0.py`)

Dataset. `build_dataset.py` merges every successful C++-executor run into `results/dataset.csv`: 504 runs over 85 distinct configuration+workload combinations.

Features use only what a controller knows *before* running a request:

Feature group	Columns
Configuration	precision, device, backend, cores, threads, KV precision, prefix caching, cache size
Workload	prompt tokens, output tokens, prefix-cache hit
Machine state	background CPU %, available RAM — measured just before the run

Targets: TTFT, TPOT and peak RSS.

Predictors.

- **analytical** — four physical laws from the findings, fitted per hardware placement by least squares:
- TTFT ∝ prompt tokens
- cache-hit TTFT ≈ constant
- TPOT ≈ linear in context
- memory ≈ linear in tokens

Groups fall back to coarser ones when a configuration is unseen. - **gbm** — gradient-boosted trees (`HistGradientBoostingRegressor`) on log targets.

Evaluation. 5-fold `GroupKFold` holding out *whole configurations*, so every prediction is for a configuration+workload the model never saw. The **noise floor** predicts each run from the median of the *other* runs of the same configuration — the best any predictor could do without knowing the run's own noise.

Finding 15 — Simple physical laws predict as well as ML, close to the noise floor

Target	Predictor	Mean APE	Median APE	Within ±20%
TTFT	noise floor	19.3%	11.7%	65.9%
	analytical	23.5%	15.7%	57.7%
	gbm	29.0%	15.5%	59.9%
	gbm, no machine state	33.1%	21.3%	47.0%
TPOT	noise floor	13.7%	8.4%	76.2%
	analytical	15.4%	11.2%	71.0%
	gbm	15.0%	10.3%	75.8%
Peak RSS	noise floor	0.4%	0.0%	100%
	analytical	1.1%	0.3%	100%
	gbm	3.3%	0.5%	94.0%

- **Both predictors are within ~4 percentage points of the noise floor** on TTFT and TPOT median error, and memory is predicted almost exactly. Most of the remaining error is run-to-run noise that no pre-run predictor can see.
- **The analytical model matches the ML model.** On this data, what matters is getting the physics right: proportional prefill, constant cache-hit TTFT. Model capacity adds little. That is a useful, honest result: the Twin is explainable.

- **Machine state helps the ML model's TTFT predictions:** within ±20% rises from 47% to 60% of runs when background CPU and free RAM are included.
- **A bug found by evaluation.** The first analytical version predicted a flat TTFT when a held-out configuration's group had been seen at only one prompt length, giving 10× errors (mean APE 115%). Assuming TTFT ∝ prompt length in that case fixed it.
- **Weak spots:** GPU FP16 (median APE 40–47%, little GPU data) and very short prompts (`load_time`: 32 tokens, 61–68%). More GPU and short-prompt data are needed.

Phase 6 — 3B model (Qwen2.5-Coder-3B-Instruct, INT4)

Pre-converted IR from `huggingface.co/OpenVINO/Qwen2.5-Coder-3B-Instruct-int4-ov` (1.8 GB, Apache-2.0); there was not enough RAM or disk to export it locally. Architecture: 36 layers, 16 attention heads, 2 KV heads, head dim 128, hidden 2048, MLP 11008, vocab 151,936, 32k context.

Tooling fix: Qwen's exported tokenizer ignores `max_length`/`truncation`. `orbit_run` now encodes the full text and slices the token tensor to the exact length, which gives identical results for TinyLlama.

Finding 16 — First 3B results, and a refuted hypothesis

Single runs, INT4, 32 output tokens, about 1.3–2.4 GB of RAM free:

	CPU, 256 tokens	**CPU, 1920 tokens**	**GPU, 256 tokens**	**GPU, 1920 tokens**
TTFT	6.0 s	**50.4 s**	0.97 s	9.6 s
TPOT	127 ms (7.9 tok/s)	159 ms	58 ms (17 tok/s)	106 ms
Peak RSS	3841 MiB	4222 MiB	2545 MiB	2676 MiB
Committed after warm-up	2235 MiB	2616 MiB	2461 MiB	2595 MiB

- **On CPU, a 3B model is borderline-unusable for long prompts.** A 1920-token prompt takes 50 s to first token. The GPU is 5–6× faster on TTFT and 1.5–2.2× faster on decode.
- **Memory is tight.** CPU peak RSS reached 4.2 GB while the system had only 1.3 GB free. This is the first model where OOM risk is a live constraint on this laptop.
- **Finding 7's logits hypothesis is refuted** by a prediction made before running. Full-vocabulary logits per prompt token would cost 151,936 × 4 B = 0.58 MiB per token, for ~990 MiB growth from 256 to 1920 tokens. The measurement was **382 MiB** (0.23 MiB per token).
- **Better hypothesis: MLP activation buffers.** Remove the KV share (u8: 0.011 MiB per token for TinyLlama, 0.018 for Qwen) and compare the rest with `(hidden + 2 × MLP) × 4 B`:

Model	Non-KV growth	`(hidden + 2 × MLP) × 4 B`	Ratio
TinyLlama (MLP 5632)	0.123 MiB	0.051 MiB	2.4×
Qwen2.5-3B (MLP 11008)	0.21 MiB	0.092 MiB	2.3×

The constant ratio across two architectures suggests prefill memory is dominated by a few live intermediate buffers of size `prompt_tokens × (hidden + 2 × MLP)` in f32. Still a hypothesis, but it now predicts both models, and it gives the Twin a memory feature derived from architecture.

Next for Phase 6: a proper 3B plan (n = 6, shuffled), run with as much free RAM as possible. Planned dimensions: GPU vs CPU P-cores, KV precision at long context, and prefix caching. GPU memory needs its own measurement, because process RSS under-counts iGPU buffers.

Finding 17 — 3B model: the GPU is what makes it usable (`plans/qwen3b.json`)

n = 6 per configuration, 2 shuffled rounds, 64 output tokens, unique prompts.

Qwen2.5-Coder-3B INT4	CPU P-cores, 256 tok	GPU, 256 tok	CPU P-cores, 1024 tok	GPU, 1024 tok
TTFT	5.8 s	**0.96 s**	29.0 s	**4.3 s**
Decode	8.7 tok/s	**18.9 tok/s**	7.0 tok/s	**18.4 tok/s**
Peak RSS	3838 MiB	2524 MiB	4014 MiB	2624 MiB

- **Speed:** the GPU is 6–7× faster on TTFT and 2.2–2.6× faster on decode. At 1.1B (Finding 11), the GPU was the better choice; at 3B, CPU-only gives 29 s to first token for a one-page prompt.
- **KV precision on CPU (u8 default vs f16):**
- Memory: f16 costs +18 MiB at 1024+64 tokens, against 19.1 MiB from theory (Qwen: 18 KiB per token at u8, 36 KiB at f16).
- Speed: f16 was *faster* at 1024 tokens, with TPOT 106 vs 144 ms and TTFT 25.4 vs 29.0 s. That suggests u8 dequantization costs decode time at longer contexts, matching the weak signal in Finding 6. But the u8 TPOT range was wide (95–217 ms), so it is not conclusive.
- **Explicit KV precision crashes on the GPU.** Both u8 and f16 fail on the first `generate()`:

```
[GPU] Incorrect block size for Paged Attention operation for key cache quant mode
BY_CHANNEL. Expected 20, but got 12
```

Only the plugin default works; it reports `KV_CACHE_PRECISION = dynamic`. The failing rows are recorded in `results/gpu_u8_kv_failure.jsonl` and `results/experiments/qwen3b.jsonl`. An upstream issue is drafted in `notes/upstream_issue_gpu_kv_precision.md` (not filed). For ORBIT this is a **hard constraint**: KV precision is a CPU-only setting on this stack.

Finding 18 — Long context on the GPU, and a machine stall caught by telemetry (`plans/qwen3b_long_context.json`)

Prompt	TTFT	Prefill rate	TPOT	Peak RSS
2048	8.7 s	235 tok/s	56 ms	2739 MiB
4096	19.9 s	206 tok/s	53 ms	3011 MiB
8192	53.2 s	154 tok/s	61 ms	3330 MiB

- **Prefill is superlinear at long context.** Doubling the prompt raises TTFT 2.3× (from 2k to 4k) and then 2.7× (from 4k to 8k), because attention cost grows with the square of the length. Decode slows only slightly.
- **One run stalled for 9.8 minutes** (4096 tokens, round 2): TPOT was 8425 ms/token against ~53 ms normally, and wall time was 588 s.
- The during-run telemetry shows the whole machine was idle: own CPU 0%, background CPU 2%, against 11–14% in the neighbouring runs.
- This points to the iGPU or system being put into a low-power state mid-run (for example a display timeout), not to anything in the model. It happened despite the runner holding a Windows "system required" request.
- **Rule adopted:** `build_dataset.py` drops runs whose TTFT or TPOT exceeds 5× their own configuration's median. It drops exactly this one run. Ordinary noise stays within 2×.

Finding 19 — On new data, the physical model generalizes and the trees do not

The Twin was re-run on the extended dataset: 557 runs, 94 configurations, now including 3B.

Target	Noise floor	Analytical	GBM
TTFT, median APE	11.4%	**15.7%**	18.0%
TTFT, mean APE	18.8%	**24.5%**	393.7%
TPOT, median APE	8.1%	11.2%	**9.3%**
Peak RSS, median APE	0.1%	**0.3%**	0.4%

- **The GBM's TTFT collapses on prefix-cache hits.** For held-out *repeated* 1920-token prompts it predicted ~13 s against an actual 58–77 ms (errors of 17,000–22,000%). Cache hits at 512 tokens were in its training folds, but the trees learned "long prompt → slow" without learning *why* hits are fast. The analytical model encodes the mechanism (a hit has near-constant TTFT) and predicted 60–69 ms.
- **On the 3B model, the analytical TTFT was 14–24% median error**, against 54–69% for the GBM.

- **Conclusion:** for a controller that has to handle configurations and workloads not in its training data, the mechanism-based Twin is more trustworthy, and it is explainable. `orbit.py` uses the analytical Twin. The GBM remains slightly better on TPOT within the training distribution.

Phase 7 — Deployment and real-world factors

Finding 20 — On battery, inference is ~10× slower

The same scripted 4-turn chat (TinyLlama INT4, CPU, `orbit_chat`) was run while the laptop was unplugged, at 29% battery on the Balanced power plan. Raw rows: `results/chat/prefix_cache_*_BATTERY.jsonl`.

	On battery	**On AC (Phase 4/5 medians)**
TPOT	350–540 ms	~46 ms
TTFT, 26-token cold prompt	~31 s	~0.3 s

- Even so, prefix caching still worked: turn 3 TTFT was 3.3 s with caching on and 25.4 s with it off.
- **Action:** `orbit_run` and `orbit_chat` now record `on_ac_power` and `battery_percent`. `orbit_chat` warns at start-up on battery. `run_experiment.py` and `run_chat_study.py` refuse to benchmark on battery unless `--allow-battery` is given.
- For deployment, power source is a first-order performance factor on a laptop. A controller should treat "on battery" as a different machine.

Finding 22 — Keeping models resident, and contention on the shared iGPU

Residency (`residency_experiment.py`, `results/experiments/residency.jsonl`). INT4 TinyLlama on the iGPU, 1024-token prompts:

	Fresh process per request (`orbit_run`)	**Long-lived worker (`orbit_worker`)**
TTFT	2447 ms	**1650 ms**
TPOT	36.2 ms	**31.0 ms**

- In the worker, a load costs 3.45 s for INT4 TinyLlama and 8.6 s for Qwen-3B INT4 (GPU), paid only when the model changes.
- A resident model is faster per request *and* avoids the reload. A real deployment should keep the model loaded.

GPU contention (`plans/gpu_contention_{calm,busy}.json`). A second LLM generated continuously on the iGPU (`noise_gpu.py`) during the "busy" runs. Medians, n = 6:

INT4, 1024-token prompt	Calm TTFT / TPOT	Busy TTFT / TPOT
CPU default	12.8 s / 82 ms	23.6 s / 159 ms
CPU P-cores	10.3 s / 54 ms	16.5 s / 123 ms
iGPU	**1.5 s / 29 ms**	**2.5 s / 47 ms**

- Contention slows the GPU path 1.6–2.1×, but the GPU still beats every CPU configuration.
- The CPU paths also slow by ~1.6–2.3× even though the noise ran on the GPU. The iGPU shares system memory bandwidth (and host-side CPU work) with the CPU, so on an integrated GPU there is no fully isolated device.

Finding 21 — Cache eviction: no benefit in chat, and it crashes with prefix caching

`run_chat_study.py` replays `conversations/long_recall.txt` through `orbit_chat`: 12 turns, 80-token answers. Turn 1 states two facts, and turn 12 asks for them back. Raw rows: `results/chat_eviction.jsonl`. All runs on AC power.

- **Prefix caching + eviction crash.** With both on, generation fails once eviction begins: `Check 'is_all_free' failed ... blocks across layers must be freed simultaneously` (`block_manager.hpp:371`). It happened after 3 turns at `32:64:256` and 5 turns at `32:128:512`, on CPU and GPU. Rows: `results/chat/eviction_with_prefix_caching_CRASHED.jsonl`. Draft issue: `notes/upstream_issue_eviction_prefix_caching.md`. `orbit_chat` now turns prefix caching off when `--evict` is given.
- **Results at turn 12** (prompt ~1000–1200 tokens):

Model	Configuration	TTFT	Final RSS	TPOT, turns 9–12	Recall
Qwen-3B, GPU	prefix on	446 ms	2524 MiB	55.6 ms	yes
	prefix off	5389 ms	2604 MiB	57.6 ms	yes
	evict 512	4979 ms	2847 MiB	57.9 ms	yes
	evict 256	5218 ms	2844 MiB	65.1 ms	yes
TinyLlama INT4, CPU	prefix on	338 ms	1337 MiB	53.3 ms	no
	prefix off	7753 ms	1494 MiB	49.8 ms	no

Model	Configuration	TTFT	Final RSS	TPOT, turns 9–12	Recall
	evict 512	7700 ms	1500 MiB	50.2 ms	no
	evict 256	7650 ms	1502 MiB	47.5 ms	no

- **Prefix caching is the feature that matters for chat:** 12–23× faster TTFT at turn 12, and less memory, because only the new tokens need prefill activations.
- **Eviction gave no gain.** Each turn still prefills the whole prompt, and eviction acts during generation. With 80-token answers and a KV cache of only ~15–40 MiB at this length, there is little for it to save. On the GPU it used ~240 MiB more.
- **Recall:** Qwen recalled both facts under every budget, but that does not prove the facts' tokens were retained: the first answer token sees the whole prompt, and eviction keeps high-attention blocks. TinyLlama's failure is the model's, since it fails with eviction off too.
- **Not tested:** very long single generations, which is what eviction is designed for.

Finding 23 — Quantization costs almost no quality

`quality_eval.py` measures bits per byte (BPB) on a fixed 34 KB corpus of OpenVINO GenAI documentation and Python code (`quality/corpus/`). BPB is the model's surprise at real text. Lower is better, and normalizing by bytes makes it comparable across tokenizers. Results: `results/quality.json`.

Model	BPB	Prose	Code
TinyLlama FP16	0.6922	0.8709	0.6234
TinyLlama INT8	0.6922	0.8709	0.6233
TinyLlama INT4	0.7017	0.8790	0.6333
Qwen2.5-Coder-3B INT4	0.5521	0.7422	0.4788

- INT8 shows no measurable loss against FP16, and INT4 is 1.4% worse.
- Model size matters far more than precision: 3B at INT4 is 20% better than 1.1B at FP16.
- BPB is a proxy for model quality, not a measure of task accuracy. The corpus is technical English and Python, which favours a code-focused model such as Qwen-Coder.

Finding 24 — The default prefix cache keeps only the latest prompt, and the ORBIT table was wrong

The ORBIT table compared different requests. ORBIT refused 6 of 24 requests, and those rows were left out of its means but stayed in static's. On the 18 requests both served, mean TTFT is 1.75 s vs 1.74 s, not 1.75 s vs 2.35 s. Outcomes matched on 22 of 24 requests, and ORBIT chose static's configuration for 15 of the 18 it served, so more requests from these scenarios would not separate them. Source: `results/evaluation_runs_worker.jsonl`, excluding its first 9 rows, which are an earlier partial run.

That file has no refusal rows, because refusals were not logged then. `evaluate_controller.py` now logs refusals and compares systems only on requests they all served.

Follow-ups never hit the prefix cache. All 6 static follow-ups took 3.8–4.5 s, about the same as a cold 1920-token prompt, although the log says `cache_hit: true`. That field records only that the prompt had been sent before.

`prefix_overwrite_test.py` sends prompts through one `orbit_worker` pipeline (TinyLlama INT4, iGPU, 1920-token prompts). TTFT in ms, 1 repeat:

Step	Dynamic pool (default)	Fixed 1 GB pool
A, first time	3995	3002
A, immediate repeat	22	19
B, another 1920-token prompt	2421	2403
A, after B	**2411**	**21**
C, a 64-token prompt	158	156
A, after C	**2408**	**22**

- This run was on battery, so it shows the effect, not final numbers. Raw rows: `results/chat/prefix_overwrite_BATTERY.jsonl`.
- In the default dynamic pool, any request between two repeats loses the prefix, even a 64-token one. A fixed pool keeps it, and a repeat is then about 115× faster.
- Likely mechanism: blocks freed by a finished request can be reused, least recently used first (`block_manager.hpp:143`). If matching runs from the start of the prompt, losing the first blocks would lose the whole prefix. That last step is an inference.
- Chat in `orbit_chat` is not affected while one conversation runs, because each turn extends the same prefix. Several conversations sharing one pipeline would be.
- ORBIT also refused every follow-up, so it never warmed its own cache.

Finding 25 — Task-level quality of the 3B model: INT4 shows no detectable loss

`colab/humaneval_quality.ipynb` generated answers to all 164 HumanEval problems with Qwen2.5-Coder-3B at FP16, INT8 and INT4, then ran the official tests. The scoring harness had to pass every official solution before the notebook would score the models. Raw rows: `results/quality_humaneval/`. Summary: `python summarize_humaneval.py`.

Method: greedy decoding, up to 512 new tokens, paged attention, `f32` execution precision, default CPU KV cache precision, on a Colab CPU with OpenVINO 2026.3. Model-written code ran only after Google Drive was unmounted, in a separate process with time, memory and file limits.

Model	Passed	95% CI	Failed	Hit token limit
FP16	137/164 (83.5%)	77.1–88.4%	27	2
INT8	137/164 (83.5%)	77.1–88.4%	27	2
INT4	135/164 (82.3%)	75.8–87.4%	29	3

Comparison	Only first passed	Only second passed	Difference, 95% CI	McNemar p	Identical answers
FP16 vs INT8	2	2	−2.4 to +2.4 points	1.000	137
FP16 vs INT4	8	6	−3.2 to +5.7 points	0.791	47
INT8 vs INT4	9	7	−3.6 to +6.0 points	0.804	44

- **INT4 keeps the task-level quality** within this test's resolution. The data rules out a loss above about 5.7 points, not a small one.
- **Agrees with bits per byte:** on TinyLlama, INT8 matched FP16 and INT4 was 1.4% worse.
- **INT4 changes answers much more than INT8 does.** It matched FP16 exactly on 47 problems against INT8's 137, yet the pass rate barely moved. Greedy output similarity would overstate the practical damage, so pass/fail is the better measure here.
- The INT4 model was the same file as the laptop's: the Hugging Face commit and the SHA-256 of `openvino_model.bin` match. Colab's tokens/s are not used anywhere, because the models ran in separate sessions and possibly on different hardware.
- Pass rates depend on this prompt, so they are not comparable with published scores.

Finding 26 — How many prompts a KV pool keeps: an exact, predictable boundary

Mechanism, from the OpenVINO GenAI source (`bd8d6542`): - `scheduler.hpp` sizes the default dynamic pool for the requests that are active, capped at prompt + max_new_tokens. Prompts kept for reuse are not counted. - `block_manager.hpp` gives a new request free blocks first. Once none are left, it reuses cached blocks least recently used first (`get_lru_block_to_overwrite`). - `get_full_prefix_restore_plan` matches a cached prompt block by block from the start, and stops at the first missing block. - Block size is 32 tokens on CPU and 16 on GPU (`kv_cache_manager.hpp:97`).

Prediction: K conversations of T tokens need K × ⌈T / block size⌉ blocks. Taking turns is the worst case for least-recently-used reuse, so one block short should lose every conversation.

Test: `prefix_pool_study.py`, using the new `--kv-blocks` option in `orbit_run` and `orbit_worker`, which sets `SchedulerConfig::num_kv_blocks`. TinyLlama INT4, 1920-token prompts, 16 output tokens. A warm pass c1…cK, then a measured pass c1…cK. Raw rows: `results/experiments/prefix_pool_boundary.jsonl`.

Device	Blocks per conversation	Pool (blocks)	K	Measured pass
GPU	121	dynamic	1	hit (21 ms)
GPU	121	119, 120	1	generation fails (pool smaller than the prompt)
GPU	121	121, 122	1	hit
GPU	121	dynamic, 240, 241	2	all miss (4.0–4.3 s)
GPU	121	242, 243	2	all hit (21–28 ms)
GPU	121	dynamic, 482, 483	4	all miss (3.8–4.1 s)
GPU	121	484, 485	4	all hit (19–28 ms)
CPU	61	dynamic, 61	1	hit (65 ms)
CPU	61	60	1	generation fails
CPU	61	dynamic, 121	2	all miss (14.0–17.5 s)
CPU	61	122	2	all hit (63–76 ms)

- **The prediction held at every boundary**, on both devices.
- The power state changed during the CPU run, and the GPU run was on battery. Hits and misses do not depend on power, but these timings should not be compared with AC runs.
- **Memory:** process counters cannot isolate the pool. The iGPU pool is in shared memory, and the worker's committed memory drifts across runs. From the KV formula, a CPU u8 block is 22 layers × 2 × 4 heads × 64 × 32 tokens × 1 byte = 352 KiB, so a 1920-token conversation is about 21 MiB.
- A pool smaller than one prompt fails at generation with `Check 'status != GenerationStatus::IGNORED'`, not with a clear error.
- **ORBIT changes that follow:**
- `orbit.PrefixRetention` models the pool with these rules.
- The evaluation records a timing-based `measured_hit` next to the predicted one.
- Online correction no longer learns TTFT from a run whose hit or miss went against the prediction. Before, a predicted hit that missed taught ORBIT that hits are slow.

Mains-power rerun (2026-11-09, other apps closed). Raw rows: `results/experiments/prefix_pool.jsonl`, `results/experiments/prefix_pool_margin.jsonl` and `results/chat/prefix_overwrite.jsonl`. These replace the battery timings above.

Pool	GPU hits	CPU hits	Median TTFT, hit / miss
Exact (K × need; K = 1, 2, 4, 8 GPU; 1, 2, 4 CPU)	45 / 45	13 / 14	GPU 22 ms / 3.8 s; CPU 62 ms / 16 s
One block short	0 / 42	0 / 12	—
Dynamic, K ≥ 2	0 / 42	0 / 12	—
CPU, K = 2: exact again (122)	—	10 / 10	—
CPU, K = 2: one or two spare blocks (123, 124)	—	20 / 20	—

- **One miss at the exact CPU boundary,** in 24 exact-fit CPU requests. It was the second of two conversations at 122 blocks, in one repeat of seven.
- **Likely cause, from the source; not proven:**
- Blocks taken from the free pool keep their creation timestamp (`allocate_block(hash, …)` does not reset it).
- `get_lru_block_to_overwrite` takes `std::min_element` over a map ordered by hash, so when timestamps tie, the block chosen depends on hash order.
- The CPU conversation ends in a half-filled block (1936 / 32 = 60.5), while the GPU's is exactly 121 full blocks. That may be why only CPU showed the miss.
- **Recommendation:** one spare block per conversation (352 KiB on CPU). It was never seen to miss, but 20 requests are too few to prove it.
- **The A, A, B, A test on mains power,** 3 repeats per device:

Step	GPU, dynamic	GPU, 1 GB	CPU, dynamic	CPU, 1 GB
A repeated immediately	33 ms	24 ms	67 ms	70 ms
A after a long prompt	4508 ms	24 ms	16362 ms	62 ms
A after a 64-token prompt	4561 ms	23 ms	16308 ms	59 ms

- **Battery vs mains, iGPU:** cold 1920-token TTFT on the iGPU was about 2.4 s in the battery diagnostic but 3.8–4.5 s here. This run had Docker and WSL running, and the earlier October evaluation on AC also measured 3.7–3.9 s. So the battery number is the outlier, and none of the conclusions depend on it.

Finding 27 — The two OpenVINO problems: one reproduced minimally, one gone

Search first (2026-11-09). The GitHub search API, over all openvinotoolkit repositories, found no existing report of either error. It did turn up openvinotoolkit/openvino#39615 (opened 2026-11-08): the same GPU `Incorrect block size … BY_CHANNEL` check, on an Arc B60 with f32 inference precision and u4 KV cache.

Eviction + prefix caching: reproduced in a standalone program. `cpp/repro/repro_eviction_prefix.cpp` is about 60 lines: `ChatHistory`, 80 tokens per turn, `CacheEvictionConfig(32, 64, 256)`, TinyLlama INT4 on CPU. - With prefix caching on, turn 4 fails with `blocks across layers must be freed simultaneously`, followed by `BlockAllocator leaked blocks` errors. - With prefix caching off, all 8 turns succeed. - Draft: `notes/upstream_issue_eviction_prefix_caching.md`, ready to file once it has been checked on the latest release.

GPU + explicit KV precision: no longer reproducible. The October crash rows were all Qwen-3B on the iGPU with explicit u8 or f16. None were TinyLlama, so the earlier draft's claim that it was also seen with TinyLlama was wrong, and it has been corrected. Rerun today: - `orbit_run`, Qwen-3B INT4 on GPU, u8 and f16, 256 and 1024 tokens, shared model cache: all 4 succeed; - `cpp/repro/repro_gpu_kv_precision.cpp` with Qwen-3B, u8 and default: both succeed; - TinyLlama INT4 with u8 and f16, with no cache, a fresh cache, the shared cache and a second load: all succeed.

The OpenVINO install (25 Aug), the GenAI build (`bd8d6542`), the driver (32.0.101.5542) and the code that passes the precision are all unchanged. Today's u8 runs compiled new entries in `ov_cache/gpu`, which suggests that on 2026-10-06 a cached model was loaded instead. That is unconfirmed. **The cause is unknown, so the draft is marked "do not file".**

Source reading for the record: in OpenVINO 2026.3, `transformations_pipeline.cpp` sizes a BY_CHANNEL key block from the inference precision, while `paged_attention.cpp` checks against fixed formulas: `16 + 4 = 20` for 8-bit and `16 / 2 + 4 = 12` for 4-bit. "Expected 20, but got 12" is therefore an 8-bit expectation meeting a 4-bit-sized block.

Consequence: GPU KV-cache precision can be studied after all.

> **Correction (Finding 30):** this rerun did not test u8 or f16 on the GPU. With the shared `CACHE_DIR`, the GPU model cache returns the first compiled blob whatever KV precision is requested, so these runs used the cached default layout (4-bit).

Update, same day: still present in the latest release. The Python version of the repro, `cpp/repro/repro_eviction_prefix.py`, was run with OpenVINO GenAI 2026.4.1.0 (`ab01d1771e4`, pip) in a throwaway environment. Turn 4 fails with the same message and the same `BlockAllocator leaked blocks` errors, and with prefix caching off all 8 turns pass. A control run on the current 2026.3 install failed identically. The report is ready to paste: `notes/issue_eviction_prefix_caching_TO_FILE.md`.

Finding 28 — TinyLlama on HumanEval: 6–7%, at every precision

Same notebook and settings as Finding 25 (`colab/humaneval_tinyllama.ipynb`), on the laptop's own TinyLlama files, uploaded to Drive and checked by the SHA-256 of `openvino_model.bin`. Raw rows: `results/quality_humaneval_tinyllama/`. Summary: `python summarize_humaneval.py results/quality_humaneval_tinyllama`.

TinyLlama-1.1B	Passed	95% CI	Hit 512-token limit
FP16	10/164 (6.1%)	3.3–10.9%	35
INT8	11/164 (6.7%)	3.8–11.6%	32

TinyLlama-1.1B	Passed	95% CI	Hit 512-token limit
INT4	12/164 (7.3%)	4.2–12.4%	40

- **Quantization does not measurably change it:** FP16 vs INT4 split 4 / 6 problems, p = 0.75. Pass rates are so low that this says little about INT4's cost.
- **How it fails:** about 45 of 164 failures per model are `NameError`, and most of those (33–45) come from one habit. The model writes `for I in range(...)`, then uses `i`. Other failures are missing imports and wrong results. These are model errors, not scoring errors: the answers were read, and the official solutions pass the same harness.
- **For ORBIT's code-quality floor (50%):** even if every `NameError` answer had passed, TinyLlama would reach at most 56–63 of 164 (34–38%). It is barred from code requests in every case. Re-running the v2 dry run with the measured scores changes nothing.
- **Model size matters far more than precision:** Qwen2.5-Coder-3B passes 82–84% at every precision, TinyLlama 6–7%.

Finding 29 — Evaluation v2 on real data: ORBIT beats both fixed baselines, and ties a simple rule

The pre-registered evaluation of `notes/evaluation_v2_plan.md`, run overnight on 2026-11-11 on mains power (`run_overnight.ps1`, `evaluate_v2.py`). - **Workload:** 300 test requests with real lengths: 145 chat turns (oasst1), 81 one-shot and 74 code requests (Azure 2023 trace). - **Tuning:** the best fixed configuration was chosen on a separate 300-request tuning trace. - **How it ran:** one system was loaded at a time, in blocks of 25 requests (plan section 12). - **Raw data:** `results/evaluation_v2/`. Analysis: `analysis_corrected.txt`, from `python evaluate_v2.py analyze results/evaluation_v2/pass_{A,B,C}.jsonl --corrections results/evaluation_v2/corrections.jsonl --sweep results/evaluation_v2/sweep_tuning.jsonl`.

A measurement fault, found and corrected first. Five requests reported a ~50 ms time to first token on a 4,096-token prompt for the 3B model, against ~23 s for every other prompt that long. Re-running them one at a time showed real prefix-cache hits. - **Cause:** the Qwen tokenizer IR truncates from the **left** at 32,768 tokens (`Truncate` with `max_length=32768` and side `left`, in `openvino_tokenizer.xml`). Our synthetic prompt text for targets above about 4,090 tokens was longer than that. So the request tag at its start was cut off, and two prompts of the same length became identical (`prompt_uniqueness_check.py`). - **TinyLlama's prompts were always distinct,** although its IR also lists a left truncation, at 2,047 tokens. Earlier results such as Finding 26 are unaffected. - **Fix:** `create_exact_token_input` builds only the text it needs and checks that the tag survives. Below the limit it gives the same token ids as before (`cpp/repro/check_prompt_ids.cpp`). - **Checked after the fix:** a 4,096-token prompt with another tag now misses the cache (24.4 s); before, it hit in 47 ms. The before and after runs are `results/prompt_uniqueness_check_before_fix.jsonl` and `results/prompt_uniqueness_check.jsonl`. - **Correction:** the 9 affected rows were re-measured with unique prompts (`rerun_duplicate_prompts.py`). Each gained a TTFT violation (24–25 s). Plan section 13 records this. The numbers below are corrected; uncorrected, ORBIT and the rule each have 2 fewer violations and ORBIT vs best fixed is 14 to 0.

Service-level violations, all 300 test requests (lower is better):

System	Configuration	Chat (145)	One-shot (81)	Code (74)	Total	95% CI
OpenVINO default	TinyLlama INT8 @ CPU	13	65	74	152 (50.7%)	45.0–56.3%
Best fixed (tuned)	TinyLlama INT4 @ iGPU	0	22	74	96 (32.0%)	27.0–37.5%
Best fixed, A/A rerun	TinyLlama INT4 @ iGPU	0	21	74	95 (31.7%)	26.7–37.1%
Simple rule	3B for code or long prompts, else TinyLlama INT4 @ iGPU	0	21	63	84 (28.0%)	23.2–33.3%
ORBIT without correction	Twin + optimizer	0	25	63	88 (29.3%)	24.5–34.7%
ORBIT	all three layers	0	21	63	**84 (28.0%)**	23.2–33.3%

Paired tests (same requests):

Pair	Only first broke	Only second broke	McNemar p	Median total-time ratio (95% CI), both served
ORBIT vs best fixed (primary, interleaved in pass A)	0	12	**0.0005**	0.997 (0.988–1.005), 216 requests
ORBIT vs OpenVINO default	0	68	7×10^{-21}	0.352 (0.350–0.356), 216
ORBIT vs simple rule	1	1	1.0	0.994 (0.991–0.999), 300
ORBIT vs ORBIT without correction	1	5	0.22	1.003 (0.997–1.008), 300
Best fixed vs its A/A rerun (noise floor)	1	0	1.0	1.016 (1.009–1.022), 216

- **The primary test is significant:** ORBIT had 12 fewer violations than the tuned fixed configuration, and was never worse on any request ($p = 0.0005$).
- 11 of the 12 come from routing: short code requests that ORBIT sent to the 3B model, which met the limit. Best fixed may not serve code with TinyLlama (HumanEval 7%, Finding 28).

- The 12th (request 225) is timing noise. Both systems ran TinyLlama INT4 on the iGPU, with TTFT 1.97 s against 2.25 s at a 2 s limit. Request 8 is the same kind of borderline case, and is the A/A run's only discordant request.
- Without request 225 the result is 11 to 0, p = 0.001.
- On the 216 requests both served, ORBIT was exactly as fast (ratio 0.997).
- **ORBIT ties the simple rule** (84 vs 84, one discordant request each way), as the dry run predicted.
- On this laptop, "use the 3B model when the request needs it, else TinyLlama INT4 on the iGPU" captures the whole benefit.
- ORBIT's other choices (INT8 instead of INT4 on the iGPU, for 27 requests) changed only one outcome: request 8, at the 2 s borderline (2.07 s against the rule's 1.97 s). They made those requests slower in total time, because INT8 decodes more slowly.
- **The default export on the CPU is the worst choice:** 68 more violations than ORBIT, and 2.8× slower on the requests both served. Its chat violations are TTFT over 2 s.
- **The noise floor:** the A/A rerun differs by 1 violation and 1.6% in time, and that small time drift is statistically detectable (Wilcoxon $p = 2 \times 10^{-8}$). Cross-pass differences smaller than about 2% are therefore not claimed. That covers ORBIT vs rule (0.6%) and ORBIT vs no correction (0.3%).
- **Online correction (Layer 3) helps the predictions, not the outcome:**

•	**ORBIT**	**ORBIT without correction**
Median TTFT prediction error	14%	30%
Median decode-speed (TPOT) prediction error	5%	61%
Model switches (load time)	26 (100 s)	69 (256 s)
Violations	84	88 (p = 0.22)

Without correction, the Twin overestimates decode time on the iGPU. That makes the optimizer swap models more often. - **Most remaining violations are impossible on this hardware**, and fail for every system: - *Code:* the 3B model reaches its first token within 1.5 s only for prompts up to about 300 tokens (11 of 74 met it). The median code prompt has 1,245 tokens, and at ~5.5 ms per token on the iGPU, 4,096 tokens take ~23 s. - *One-shot:* all 21 that broke had long prompts, of at least about 1,300 tokens. 10 were too long for TinyLlama and too slow on the 3B model. The other 11 fit TinyLlama, but it needs a little over 2 s to process them on the iGPU. - *Chat:* every iGPU system met every chat limit. - **What ORBIT and the rule cost:** they keep two models resident, with a median peak of 3.5 GB against 1.1 GB for the fixed TinyLlama. Free RAM fell below 512 MiB on 36 ORBIT requests. Their TTFT matched the rule's (ratio 0.98), so paging did not distort the timings. - **The dry run predicted the ranking, not the counts:**

Test trace	**Default**	**Best fixed**	**Rule**	**ORBIT**
Twin's prediction (plan section 7)	168	112 (INT8 @ iGPU)	97	96

Test trace	Default	Best fixed	Rule	ORBIT
Measured	152	96 (INT4 @ iGPU)	84	84

- The Twin was pessimistic by 12–16 violations for every system.
- The measured sweep chose INT4 over INT8 on the iGPU (tuning trace: 112 vs 117 violations). The dry run had called these two nearly tied.
- **Prefix caching hardly mattered here.** Real chat prompts are short (median 61 tokens), and chat met its limits anyway.
- ORBIT's hit predictor covers exact repeats only. So it never predicted the partial reuse a follow-up gets from its earlier turn: follow-ups ran at 1.6 ms per prompt token, against 2.9 for first turns.
- This errs on the safe side (cold TTFT predicted), and is left as future work.

Finding 30 — KV-cache precision on the iGPU: 4-bit by default, and the model cache ignores the setting

The study Finding 27 made possible, run 2026-11-11. Model: Qwen2.5-Coder-3B INT4 on the Iris Xe iGPU, plus TinyLlama for the layout. - **Layout:** `kv_layout.py` → `results/kv_layout.json`. This repeats GenAI's own steps: `SDPAToPagedAttention`, then compile, then read the `key_cache.N` / `value_cache.N` inputs that `kv_cache_manager.hpp` sizes blocks from. - **Quality:** `cpp/orbit_generate.cpp` and `kv_quality.py` → `results/kv_quality/`. - **Memory:** `kv_memory.py`, from the Windows `GPU Process Memory` counters (`gpu_memory.py`) → `results/kv_memory.jsonl`.

1. What the iGPU actually stores. Compiled without a model cache:

Requested KV precision	Key / value element	Key block (heads, head dim, rows)	Qwen-3B	TinyLlama
default (`dynamic`)	4-bit, in u8 containers	[2, 128, 12]	**11.5 KiB/token**	7.2 KiB/token
u4 or i4	same as default	[2, 128, 12]	11.5	7.2
u8 or i8	i8	[2, 128, 20]	20.5	12.7
f16	f16	[2, 128, 16]	36.0	22.0
f32	compile fails (the GPU computes in f16)	—	—	—

- **The extra rows are the quantization parameters:** 16 tokens plus 4 for 8-bit, and 16/2 plus 4 for 4-bit. Keys are quantized per channel, values per token.
- **On the CPU the default is u8** (Qwen: 20.8 KiB/token). f32, f16 and u4 also work there; i8 and i4 are rejected.

- **The default `KV_CACHE_PRECISION = dynamic` means 4-bit on this iGPU,** on 2026.3 and on 2026.4.1. So every GPU measurement in this project ran on a 4-bit KV cache, including evaluation v2 (Finding 29). The HumanEval scores in Findings 25 and 28 were measured on CPU, with a u8 cache.

2. The GPU model cache ignores `KV_CACHE_PRECISION`. - **What happens:** with `CACHE_DIR` set, a second compile asking for a different precision returns the first compile's blob, with the first one's KV layout. Meanwhile `get_property(KV_CACHE_PRECISION)` reports the new value, as a plain string. - **Repro:** `cpp/repro/repro_gpu_cache_kv_precision.py`. It reproduces in both orders (f16 then u8, u8 then f16), on 2026.3 and on 2026.4.1, the latest release. - **No upstream report found.** The text to file is in `notes/issue_gpu_cache_kv_precision_TO_FILE.md`. - **Consequences here:** - `orbit_worker` always passes the shared `ov_cache/gpu`, so any earlier GPU run that asked for a KV precision got whatever was cached. - The Finding 27 rerun that "succeeded" with u8 and f16 on the GPU therefore did not test those precisions. - October's `Expected 20, but got 12` crash is an 8-bit block size (20) meeting a 4-bit cached block (12), which fits this mechanism. That stays a hypothesis, because the crash itself did not come back. - Every measurement below was compiled without a model cache.

3. Memory: exactly the layout. Peak shared GPU memory of a fresh process answering one prompt, 2 repeats each (identical to within 1 MiB):

Prompt + output tokens	4-bit (default)	int8	f16	int8 − 4-bit (theory)	f16 − 4-bit (theory)
1,042	2,219 MiB	2,228	2,243	+9.1 (+9.2)	+24.8 (+24.9)
2,043	2,307	2,325	2,354	+18.4 (+18.0)	+47.3 (+48.8)
3,543	2,450	2,481	2,533	+31.6 (+31.1)	+83.4 (+84.7)

- The differences match the layout within 1.5 MiB, so this counter sees the iGPU KV cache exactly. Process RSS does not.
- **The cache is small next to the weights:** at 3.5k tokens the KV cache is 40, 71 or 125 MiB, against about 2.2 GB in total. Memory grows by about 0.09 MiB per prompt token at every precision, and most of that is activations, not KV.
- **The 4-bit default saves 85 MiB at 3.5k tokens,** about 3% of the process. The saving only matters at much longer contexts or with many sequences.

4. Speed: no measurable effect. - **Prefill:** the first token took the same time at every precision. At 1k, 2k and 3.5k tokens the medians were 5.0–5.4 s, 9.8–9.9 s and 18.7–19.6 s, over 24 prompts per cell. - **Decode:** short-context decode on HumanEval was 53–58 ms/token in no consistent order (164 prompts each). Long-context decode was within noise.

5. Quality. Greedy decoding, the same prompts and settings as the Colab reference, f16 as the reference.

iGPU KV cache	HumanEval answers identical to f16	Code identical, ignoring comments	HumanEval pass rate	Long-context retrieval
4-bit (default)	99 / 164	112 / 164	not measured	72 / 72
int8	156 / 164	157 / 164	not measured	72 / 72
f16	(reference)	—	not measured	72 / 72

- **The 4-bit default changes the output of 40% of HumanEval answers;** int8 changes 5%. Where an answer differs, the median first difference is 395 characters in (4-bit) and 302 (int8).
- Whether this costs correctness needs the pass rates, which were **not measured before submission**.
- Measuring them means running the generated code, which is done only on Colab. The answers are in `results/kv_quality/kv_quality_answers.zip`, and `colab/humaneval_score_laptop.ipynb` scores them in about 10 minutes on a CPU runtime.
- Until then, the 40% figure shows that the outputs change, not that they get worse.
- **Retrieval does not separate the precisions:**
- *Setup:* a 6-digit code next to two decoys with other names, hidden at 10%, 50% or 90% depth in 1,000–3,500 tokens of real OpenAssistant text.
- *Result:* every precision answered all 72 correctly, with identical answers. The test is at its ceiling for a 3B model at 3.5k tokens, so it shows no loss, but it cannot show a small one.